

2D Convolution Accelerator - EE5332 Course Project

Nithin Ken Maran
EE23B050

April 2026

1 Architecture

The 2D convolution accelerator is implemented using line buffers to store K rows of the image (where K is the dimension of the kernel), and a sliding window, that computes the dot product of the overlap between the kernel and the line buffer's contents.

An example architecture for a 3×3 kernel and 5×5 image is shown below:

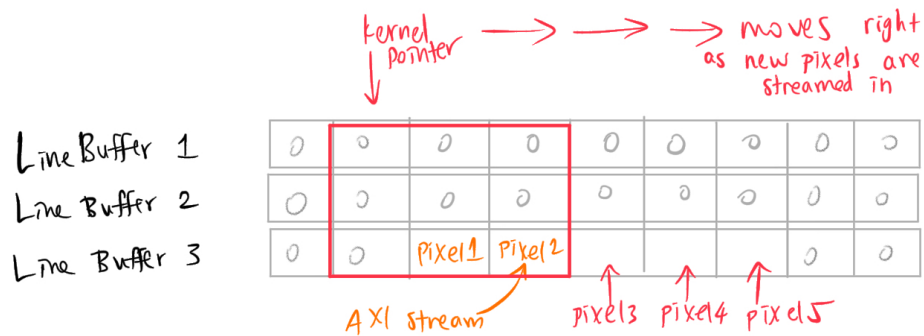


Figure 1: Kernel is implemented as a sliding window over the line buffer. A MAC loop iterates over the overlapping elements to produce the output pixel.

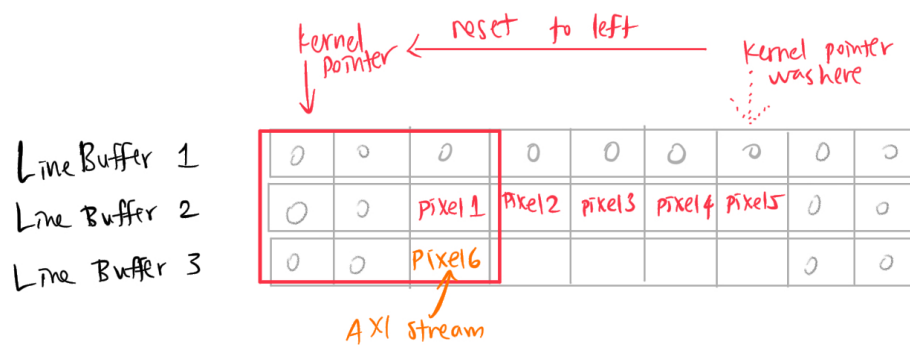


Figure 2: When an image row gets filled, the line buffers are shifted up and the sliding window is reset to the left.

2 Pareto Tradeoffs

The following is the output of my script: `read_reports.sh`, which compares the `csynth.rpt` files corresponding to 4 different directives:

- **basic**: Explicitly turn off loop pipelining and unrolling
- **Pipelined**: Pipeline all loops, and partition the line buffer arrays
- **Unroll**: Same as pipelined, but also unroll the MAC loop that computes dot product of kernel and overlapping line buffer elements
- **Aggressive**: Forces pipelining and unrolling of the outermost pixel processing loop

```
=====
HLS Synthesis Summary
=====
```

Variant	Latency	Time(ns)	DSP	BRAM	FF	LUT
basic	244614	2.446e+06	3 (1%)	2 (~0%)	1156 (1%)	1474 (2%)
pipelined	9981	9.981e+04	27 (12%)	14 (5%)	3825 (3%)	2318 (4%)
unrolled	9981	9.981e+04	27 (12%)	14 (5%)	3825 (3%)	2318 (4%)
aggressive	4766	4.766e+04	1782 (810%)	6 (2%)	149252 (140%)	63292 (118%)

2.1 Tradeoffs

1. The **basic** version uses minimal resources, but has very high latency.
2. The **pipelined** and **unrolled** versions have much lower latency than **basic**, but use more resources.
3. The **aggressive** version has the least latency, but the DSP, FF and LUT count is much larger, and cannot fit on the Pynq-Z2 board.

3 Measured Results

3.1 Correctness

```
Input image shape: (64, 64)
Top-left 5x5 of input image:
[[ 1  2  3  4  5]
 [65 66 67 68 69]
 [129 130 131 132 133]
 [193 194 195 196 197]
 [ 1  2  3  4  5]]
Kernel loaded into HLS:
[[-1. -2. -3.]
 [-4. -5. -6.]
 [-7. -8. -9.]]
Expected output shape: (66, 66)
Top-left 8x8 of expected output:
[[ -9  -26  -50  -74  -98  -122  -146  -170]
 [-591 -1131 -1618 -1657 -1696 -1735 -1774 -1813]
 [-1554 -2931 -4128 -4173 -4218 -4263 -4308 -4353]
 [-2706 -5043 -7008 -7053 -7098 -7143 -7188 -7233]
 [-1554 -2803 -3744 -3789 -3834 -3879 -3924 -3969]
 [-1170 -2099 -2784 -2829 -2874 -2919 -2964 -3009]
 [-1554 -2931 -4128 -4173 -4218 -4263 -4308 -4353]
 [-2706 -5043 -7008 -7053 -7098 -7143 -7188 -7233]]
```

Figure 3: Expected Output

```

--- DMA Streaming 2D Convolution, ap_fixed<32,16> ---
DMA transfer + accelerator time: 5.987 ms
Hardware output shape: (66, 66)
Top-left 8x8 of hardware output:
[[ -9  -26  -50  -74  -98  -122  -146  -170]
 [ -591 -1131 -1618 -1657 -1696 -1735 -1774 -1813]
 [-1554 -2931 -4128 -4173 -4218 -4263 -4308 -4353]
 [-2706 -5043 -7008 -7053 -7098 -7143 -7188 -7233]
 [-1554 -2803 -3744 -3789 -3834 -3879 -3924 -3969]
 [-1170 -2099 -2784 -2829 -2874 -2919 -2964 -3009]
 [-1554 -2931 -4128 -4173 -4218 -4263 -4308 -4353]
 [-2706 -5043 -7008 -7053 -7098 -7143 -7188 -7233]]

Difference shape: (66, 66)
Top-left 8x8 of difference:
[[0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0.]]
Max absolute difference = 0.0
Fixed-point LSB = 1.52587890625e-05
PASS: hardware output matches Python reference within one fixed-point LSB

```

(a) Hardware Output

(b) Difference between expected and hardware output

3.2 Timing

```

HW timer: 113366 cycles = 1133.660 us @ 100 MHz

```

Figure 5: AXI Timer Output

```

--- SciPy correlate2d Timing Baseline ---
Max |FPGA ap_fixed output - SciPy correlate2d| = 0.00000000
Runs: 1000
SciPy correlate2d average time: 2.197760 ms

--- FPGA ap_fixed vs SciPy Timing ---
FPGA DMA+accelerator time: 1.133660 ms
SciPy correlate2d time:    2.197760 ms
FPGA speedup vs SciPy:    1.939x

--- FPGA Throughput ---
Output pixels:            4356
Measured cycles:          113366
Pixels/cycle:             0.038424
Cycles/output pixel:      26.025
Throughput @ 100 MHz:     3.842 MPixels/s

```

Figure 6: Comparison with SciPy

Since the hardware assumes the convolution kernel is pre-flipped before loading via AXI-Lite, we must use `correlate2d` from SciPy instead of `convolve2d`.

As seen above, the hardware accelerator gives a 1.94x speedup over the SciPy version running on ARM.